

Component Modelling & Architectural Pattern Selection

Case Study: Inter-City Freight Load Matching Marketplace (ICFLMM)

Name: Thijil Thejaswi M N SRN: PES1UG24AM412

Section: 5G

1. Objective

The primary objective of Laboratory Experiment 3 is to evaluate alternative software architectural styles, select the most appropriate architecture for a complex distributed logistics platform, and develop a comprehensive UML 2.0 Component Diagram with formal interface contracts. In previous laboratory sessions, we established the system's formal Software Requirements Specification (Lab 1) and transformed those requirements into an Agile Scrum product backlog with simulated sprint executions in Jira (Lab 2). In Lab 3, our engineering focus shifts from requirements analysis to high-level systems design and Component-Based Software Engineering (CBSE).

Specifically, this laboratory exercise aims to:

- Compare and contrast three foundational architectural patterns—Layered (N-Tier), Microservices (Service-Oriented), and Client-Server—relative to real-world operational constraints and non-functional requirements.
- Formulate a rigorous, scenario-grounded technical justification for selecting a Microservices Architecture for the Inter-City Freight Load Matching Marketplace (ICFLMM).
- Decompose system capabilities into exactly five cohesive, loosely coupled microservice components adhering to black-box abstraction.
- Model formal provided interfaces (ball notation) and required interfaces (socket notation) with explicit assembly connectors.
- Analyze and document the critical security enforcement and performance optimization mechanisms inherent to the chosen architecture.

2. Introduction to Component Modelling & Software Architecture

Software architecture serves as the structural foundation of any software-intensive system. It defines the major computational components, their externally visible properties, and the relationships and communication protocols connecting them. In modern software engineering, Component-Based Software Engineering (CBSE) emphasizes the design of systems as assemblies of autonomous, replaceable, and executable software modules whose internal implementation details are strictly encapsulated.

A software component represents an independent unit of deployment and composition. Components interact exclusively through well-defined contractual interfaces, preventing architectural degradation and hidden dependencies. In UML 2.0, component modelling utilizes precise graphical notations:

- **Component Element:** Represented as a classifier rectangle bearing the «component» stereotype and the standard component icon (a rectangle with two protruding horizontal tabs).
- **Provided Interface (Ball / Lollipop Notation):** Denotes a service or set of operations that a component implements and exposes for consumption by other system modules.
- **Required Interface (Socket / Semicircle Notation):** Denotes a service or dependency that a component requires from another entity in order to execute its business functions.
- **Assembly Connector:** The visual pairing where a provided interface ball fits seamlessly within a required interface socket, signifying a direct functional dependency binding.
- **Ports:** Small square anchors positioned on the component boundary that group related interfaces and establish explicit interaction points with the external environment.

3. Project Overview: Inter-City Freight Load Matching Marketplace (ICFLMM)

The Inter-City Freight Load Matching Marketplace (ICFLMM) is an enterprise-grade B2B freight brokerage exchange designed to connect commercial shippers (e.g., manufacturers, agricultural distributors, and retail aggregators) with freight trucking carriers (fleet operators and independent owner-operators) across long-haul transit corridors. The platform addresses long-standing inefficiencies in the traditional road freight industry, including high intermediary brokerage margins (10%–20%), acute freight underutilization resulting in empty return trips ('deadhead miles'), and widespread payment insecurity where carriers wait up to 90 days for paper-based Proof-of-Delivery (POD) clearance.

The core functionality encompasses cargo load listing, multi-parameter corridor filtering, real-time carrier bidding, contract award locking, milestone dispatch tracking, digital consignee POD verification, automated milestone escrow settlement, and administrative dispute arbitration. These functions must strictly satisfy two critical Non-Functional Requirements defined in Lab 1:

- **NFR-001 (Performance):** The platform shall support a minimum of 5,000 concurrent users and respond to all search, load posting, and bid-submission requests within 3 seconds under peak traffic conditions.
- **NFR-002 (Security):** All payment data, banking credentials, and personally identifiable information (PII) shall be encrypted using AES-256 at rest and TLS 1.3 in transit, backed by rigorous Role-Based Access Control (RBAC).

4. Architectural Pattern Evaluation

To identify the optimal architectural paradigm for ICFLMM, three candidate patterns were systematically evaluated against the system's functional scope, transaction velocity, and quality attribute constraints.

4.1 Layered (N-Tier) Architecture

In a Layered Architecture, the application is organized into horizontal strata: Presentation Layer, Business Logic Layer, Persistence Layer, and Database. While this model enforces a clear separation of technical concerns and is straightforward to develop initially, it exhibits severe drawbacks for ICFLMM. The primary deficiency is monolithic coupling: high-frequency read queries (carriers searching corridor listings) compete for shared application server threads and database connections with sensitive, stateful financial operations (escrow disbursements). Scaling the system to meet NFR-001 requires duplicating the entire monolithic stack, leading to inefficient compute utilization. Furthermore, a failure in a long-running background worker (such as the 72-hour auto-approval escrow timer) can exhaust thread pools, destabilizing the entire marketplace.

4.2 Client-Server Architecture

A centralized Client-Server model utilizes smart or thin client applications (web/mobile portals) interacting directly with a centralized server cluster and database backend. While conceptually simple and offering unified transactional integrity, it creates a severe single point of failure and bottleneck. In the freight marketplace, heavy data processing would saturate server bandwidth, degrading API responsiveness for concurrent bidding carriers. Moreover, security cannot be properly compartmentalized: compromise of the centralized server exposes both public load catalogs and confidential escrow accounts simultaneously.

4.3 Microservices (Service-Oriented) Architecture

Microservices architecture structures the system as a collection of autonomous, loosely coupled, fine-grained services organized strictly around business domains. Each microservice encapsulates its own data store and communicates via lightweight interfaces. For ICFLMM, this pattern provides decisive advantages: the Load Management component can auto-scale horizontally to absorb surges in corridor searches, the Bid & Matching component can maintain isolated transactional locks, and the Escrow Payment component can reside within a fortified, private network subnet subject to stringent financial isolation. This guarantees high resilience, independent deployment, and targeted performance scaling.

5. Architectural Comparison Table

Table 1: Architectural Pattern Comparative Evaluation for ICFLMM

Evaluation Dimension	Layered Architecture	Client-Server Architecture	Microservices Architecture (Selected)
Modularity & Cohesion	Layer-cohesive; business logic remains coupled across domains.	Low; client-server separation only, monolithic backend.	High; fine-grained business domain cohesion and clear black-box boundaries.
Horizontal Scalability	Poor; entire monolithic stack must be duplicated to scale one feature.	Limited; database and application server form severe bottlenecks.	Exceptional; targeted horizontal auto-scaling per service (NFR-001).
Fault Isolation & Resilience	Low; uncaught exceptions or thread exhaustion crash the application.	Poor; central server failure terminates all marketplace operations.	High; failure in payment gateway or escrow timers is quarantined.
Security Isolation	Uniform perimeter; single breach compromises financial and load data.	Centralized perimeter; no internal micro-segmentation.	Defense-in-depth; escrow vault isolated behind private subnet (NFR-002).
Data Storage Flexibility	Rigid; typically constrained to a single shared relational database.	Uniform; single database engine manages spatial and ledger data.	Flexible; purpose-fit storage optimized for each service's workload.
Deployment Independence	Monolithic; any minor bugfix mandates full system redeployment.	Coordinated; server updates require system-wide maintenance windows.	Autonomous; services can be independently updated and deployed.
Suitability for ICFLMM	Sub-optimal; fails to handle divergent workload and security profiles.	Unsuitable; severe scalability ceiling under 5,000 concurrent users.	Optimal; directly satisfies all functional workflows and NFR constraints.

6. Selected Architecture: Microservices Architecture

"We chose a Microservices Architecture for the Inter-City Freight Load Matching Marketplace (ICFLMM)."

The selection of Microservices Architecture is directly rooted in the empirical findings of our previous lab sessions. In Lab 2's Agile sprint simulations, Epic 1 (Marketplace Operations, 23

points) exhibited high-frequency, user-interactive transaction patterns, whereas Epic 2 (Delivery, Escrow & Disputes, 49 points) was dominated by asynchronous, long-running workflows with complex banking integration. By decoupling these capabilities into autonomous microservices, we ensure that the high-velocity bidding exchange runs at peak performance without being hindered by downstream financial settlement latencies.

7. Component Identification & Decomposition

In strict accordance with the system requirements and Component-Based Software Engineering principles, the ICFLMM platform is decomposed into exactly five major autonomous microservice components:

7.1 Load Management

Responsible for the full lifecycle of freight consignments (FR-001, US-01, US-07). Enables commercial shippers to post new cargo loads with origin, destination, weight, dimensions, pickup dates, and delivery deadlines. Manages spatial corridor route indexing and provides published load specifications to the marketplace via the ILoadService interface.

7.2 Bid & Matching

Implements the core competitive bidding exchange and contract awarding logic (FR-002, FR-003, US-02 through US-05). Ingests monetary price quotations and transit time estimates from registered freight carriers. Executes concurrent bid ranking algorithms and locks contract awards upon shipper acceptance. Publishes binding contract terms to downstream fulfillment via the IBidService interface and coordinates escrow funding with the financial engine via the IPaymentService interface.

7.3 Shipment Tracking

Manages the transit lifecycle and shipment monitoring workflows (FR-004, US-08, US-09, US-10). Ingests driver transit milestones (Picked Up, In Transit, Arrived) and live GPS updates. Coordinates shipment state transitions and exposes transit milestones to downstream delivery verification via the ITrackingService interface.

7.4 Proof of Delivery

Dedicated to consignee delivery verification and legal handover workflows (FR-004, US-11). Captures digitally signed consignee Proof-of-Delivery (POD) records, delivery photo documentation, and destination OTP verification. Once delivery sign-off is confirmed, it emits verified delivery completion signals to the financial engine via the IPODService interface.

7.5 Escrow Payment

Acts as the platform's trusted financial settlement engine (FR-005, US-12, US-15, US-16). Safely holds milestone deposits in escrow upon contract award via IPaymentService. Operates an automated 72-hour auto-approval background timer that disburses net carrier payments upon receiving verified delivery confirmation through the IPODService interface or upon review-window expiration without dispute.

8. Component Interface Definitions & Method Contracts

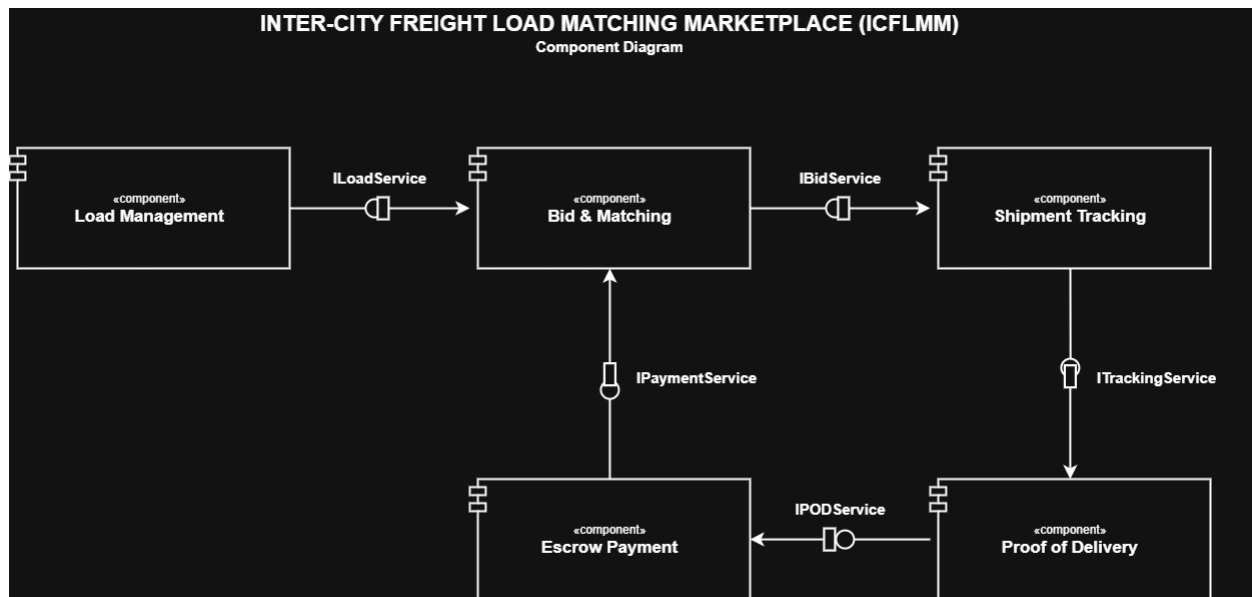
Inter-component interactions across the five microservices are strictly mediated by five formal contractual interfaces. Table 2 specifies each interface, its providing component (ball notation), its requiring component (socket notation), and its core operational methods.

Table 2: Component Interface Contracts and Method Signatures

Interface Name	Provided By (Ball)	Required By (Socket)	Key Method Signatures
ILoadService	Load Management	Bid & Matching	createLoad(specs) getLoadDetails(loadId) searchLoads(criteria) updateLoadStatus(loadId, status)
IBidService	Bid & Matching	Shipment Tracking	submitBid(carrierId,loadId, quote) acceptBid(bidId,shipperId) getContractDetails(contractId) awardShipment(loadId, carrierId)
ITrackingService	Shipment Tracking	Proof of Delivery	updateLocation(shipmentId, coords) publishMilestone(shipmentId, status) getShipmentStatus(shipmentId) notifyArrival(shipmentId)
IPODService	Proof of Delivery	Escrow Payment	submitDeliveryProof(shipmentId, signature,photoUrl) verifyOTP(shipmentId,otpCode) confirmConsigneeSignoff(shipmentId) getPODStatus(shipmentId)
IPaymentService	Escrow Payment	Bid & Matching	createEscrowAccount(contractId, amount) holdDeposit(contractId, amount) releasePayment(contractId) refundDeposit(contractId)

9. UML Component Diagram

The formal UML 2.0 Component Diagram for the Inter-City Freight Load Matching Marketplace (ICFLMM) models the five core microservices and their five ball-and-socket assembly interfaces. The diagram has been generated as a clean, standalone Draw.io source file (ICFLMM_Component_Diagram.drawio) and exported to high-resolution vector PDF and PNG formats inside the LAB 3 directory. The diagram is referenced below:



1. Load Discovery Pipeline: Load Management exposes ILoadService (Ball), which is consumed by Bid & Matching (Socket). This allows carriers to discover active freight listings and enables the bid matching engine to validate cargo specifications and corridor constraints.
2. Contract Hand-off to Fulfillment: Upon shipper bid acceptance, Bid & Matching exposes IBidService (Ball), which is consumed by Shipment Tracking (Socket). This hands off the awarded shipment details to initiate dispatch coordination, driver assignment, and live transit tracking.
3. Transit to Delivery Verification: Shipment Tracking exposes ITrackingService (Ball), which is consumed by Proof of Delivery (Socket). As the carrier nears or arrives at the destination corridor, tracking milestones trigger Proof of Delivery to prepare for consignee handover and verification.
4. Verification to Settlement Trigger: Proof of Delivery exposes IPODService (Ball), which is consumed by Escrow Payment (Socket). Once the consignee completes digital sign-off and OTP verification, Proof of Delivery emits the verified sign-off signal to Escrow Payment to initiate settlement processing.
5. Escrow Funding & Account Creation: Escrow Payment exposes IPaymentService (Ball), which is consumed by Bid & Matching (Socket). When a bid is accepted and the freight contract is

awarded, Bid & Matching invokes IPaymentService to lock the shipper's freight deposit in escrow before transit commences, completing the secure architectural lifecycle loop.

10. Architectural Justification

10.1 Architectural Selection Statement

"We chose a Microservices Architecture for the Inter-City Freight Load Matching Marketplace (ICFLMM)."

10.2 Reason 1: Decoupled Domain Lifecycles & Heterogeneous Operational Profiles

The ICFLMM platform comprises distinct operational subdomains with fundamentally divergent workload characteristics. The Load Management and Bid & Matching components operate as high-velocity marketplace engines requiring rapid, low-latency concurrency to process multi-carrier quotes without state collisions. In contrast, the Escrow Payment Service is an isolated financial subsystem governed by asynchronous milestone clearance and long-running timers. A monolithic architecture would tightly couple these disparate domains, causing heavy listing queries to block financial transactions. Microservices allow each domain to be scaled, optimized, and deployed independently without system-wide regressions.

10.3 Reason 2: Fault Isolation for Asynchronous Escrow Lifecycles & Dispute Interventions

Financial settlements in ICFLMM depend on non-instantaneous workflows, including carrier delivery confirmation, consignee Proof-of-Delivery (POD) verification, and an automated 72-hour auto-approval background timer (FR-005, US-12). In Lab 2's sprint simulations, escrow release (8 points) and dispute resolution (8 points) introduced significant complexity and latency. Under a Microservices architecture, the Escrow Payment Service operates with dedicated background queue workers isolated from marketplace operations. If external banking callbacks experience latency, the failure is quarantined within the payment service; shippers can continue discovering loads and carriers can continue submitting bids unimpeded. Furthermore, when delivery exceptions or shortages are flagged, the platform executes an atomic override lock on the escrow state machine without destabilizing active shipments.

10.4 Security Advantage: Financial Micro-Segmentation & Controlled Attack Surface

Handling high-value freight contracts and banking disbursements mandates strict security boundaries (NFR-002). The chosen architecture enforces defense-in-depth where the Escrow

Payment component and Proof of Delivery verification records reside within a secure private network subnet, entirely unreachable from public Internet endpoints. Sensitive banking credentials and escrow audit ledgers are encrypted with AES-256 and accessible exclusively through authenticated internal interfaces with mutual TLS (mTLS), fulfilling strict PCI-DSS and financial regulatory compliance.

10.5 Performance Benefit: Targeted Horizontal Auto-Scaling & Low-Latency Querying

The system must support a minimum of 5,000 concurrent shippers and carriers while sustaining sub-3-second response times under peak freight procurement windows (NFR-001). Under a traditional layered architecture, scaling requires replicating the entire monolithic stack, which incurs severe memory waste and relational database connection contention. In our Microservices architecture, instances of the Load Management and Bid & Matching components scale dynamically behind load balancers to absorb surges in multi-parameter corridor searches and quote submissions, ensuring highly responsive marketplace operations.

11. Security Considerations & Threat Mitigation

The architectural design incorporates comprehensive security controls directly aligned with NFR-002 and industry regulatory standards:

- **Granular Access Control:** Strict Role-Based Access Control (RBAC) policies ensure that carriers cannot access competitor bidding records or view escrow audit ledgers, while shippers can only review bids submitted for their own listings.
- **Network Micro-Segmentation:** Core services operate within private virtual cloud subnets. Service-to-service communication is restricted to authenticated internal channels, preventing unauthorized lateral movement across components.
- **Data-at-Rest Encryption:** All sensitive financial balances, contractual agreements, and user credentials are encrypted using AES-256 in accordance with NFR-002.
- **Tamper-Evident Proof-of-Delivery:** When a carrier uploads a digital signature and delivery photos, the Shipment Tracking Service generates a SHA-256 cryptographic hash to establish an immutable audit trail, preventing fraudulent delivery claims.

12. Performance Considerations & High-Throughput Engineering

To guarantee sub-3-second transaction turnaround for 5,000 concurrent users (NFR-001), the architecture integrates several optimization techniques:

- Targeted Horizontal Auto-Scaling: High-frequency components (Load Management and Bid & Matching) scale independently during peak shipping hours without allocating excess compute to low-frequency settlement background tasks.
- Asynchronous Job Workers: The 72-hour escrow auto-approval timer is managed by an asynchronous background worker queue, ensuring that background timer evaluation never consumes transactional thread capacity.
- Independent Persistence: Each service maintains an isolated database schema optimized for its specific read/write access patterns, eliminating lock contention across disparate business domains.

13. Conclusion

In Laboratory Experiment 3, we successfully transitioned the Inter-City Freight Load Matching Marketplace (ICFLMM) from functional and Agile requirements into a robust, scalable Component-Based Software Architecture. By rigorously evaluating Layered, Client-Server, and Microservices patterns, we demonstrated why Microservices Architecture represents the optimal technical solution for a high-concurrency, multi-stakeholder freight exchange.

The resulting architecture decomposes the system into exactly five autonomous microservices connected through five formal ball-and-socket assembly interfaces. The design guarantees independent horizontal auto-scaling to satisfy NFR-001 (5,000 concurrent users, sub-3s latency) and enforces network isolation and AES-256 data protection to satisfy NFR-002. Complete bidirectional traceability has been maintained across Lab 1 (SRS and Use Cases), Lab 2 (Agile Backlog and Sprints), and Lab 3 (Component Architecture), establishing a solid and rigorous foundation for subsequent object-oriented class design and behavioral modelling.